

Mini SPL-2
Course: Design Pattern (SE-2215)
Submitted by:
Md. Irfanur Rahman (BSSE-1612)
Md. Asaduzzaman Rifat (BSSE-1693)

CV Job Matcher

Design Patterns, UML Class Diagram, and ER Diagram

Supplementary technical documentation for the final project submission

Purpose of This Document

This document pulls together the three things a reviewer usually wants to check first: which design patterns are actually in the codebase and why, what the class-level structure of those patterns looks like, and what the database schema looks like end to end. It's meant to be read alongside the main project report, not instead of it — the reasoning behind each design decision (alternatives considered, future benefits) lives there in full; this one stays close to the diagrams and keeps the commentary short.

1. Design Pattern Summary

Eight patterns are in active use. Every one of them is doing a specific, real job in the codebase — none were added just to have a longer list. The two Proxy uses in particular (guarding notice-editing and guarding the user directory) share the same interface/implementation/proxy shape, which is a small piece of evidence that the pattern was solving a recurring problem rather than being reached for once and never again.

Pattern	Class(es)	What it's doing here
Builder	CvBuilder	Assembles a Cv (several optional, repeatable sections) step by step and validates once, at build().
Strategy	ScoringStrategy → KeywordOverlapStrategy, TfidfCosineStrategy	MatchingService/JobProcessor depend only on score(cv, notice) — the algorithm behind it is swappable.

Template Method	JobProcessor	process() fixes the score → rank → analyze-gaps skeleton; scoreStep() delegates to Strategy.
Chain of Responsibility	GapCheckHandler → SkillGapHandler, EducationGapHandler, ExperienceGapHandler	Each gap check is independent; new checks plug into the chain without touching the others.
Observer	MatchObserver → HighMatchNotifier	MatchingService notifies listeners on a HIGH match without knowing what they'll do about it.
Proxy	AvailabilityService → AvailabilityServiceImpl / AvailabilityServiceProxy; same shape for UserDirectoryService	The UI only ever calls the proxy; the proxy is the one place “admin only” is enforced, used twice for two different admin-only surfaces.
State	NoticeState → OpenState, PartiallyFilledState, ClosedState	A notice's remaining headcount changes what it's allowed to do (rankable / visible); state objects encapsulate that instead of scattered if-checks.
Singleton	DatabaseManager	One shared SQLite connection and one schema-initialization step for the whole application.

2. UML Class Diagram

Split across two figures for readability. Dashed arrows with a hollow triangle head mean “realizes/implements” an interface; solid arrows with a hollow triangle head mean “extends” a class; thin dashed arrows labeled “uses” are ordinary dependencies (one class calling another, not an inheritance relationship). Only the attributes/methods relevant to the pattern are shown — this is a design-pattern-focused view, not a full reverse-engineered class dump.

UML Class Diagram (1/2) — Strategy, Proxy, Observer, and State

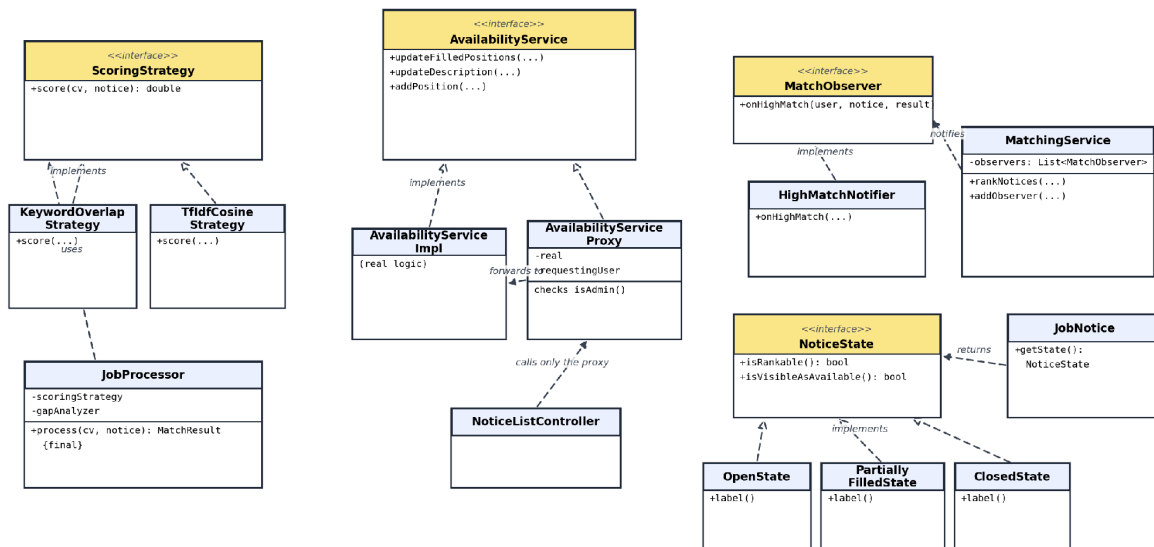


Figure 1 — Strategy (scoring), Proxy (admin-only availability edits), Observer (high-match notification), and State (notice lifecycle).

UML Class Diagram (2/2) — Template Method, Chain of Responsibility, Builder, Singleton

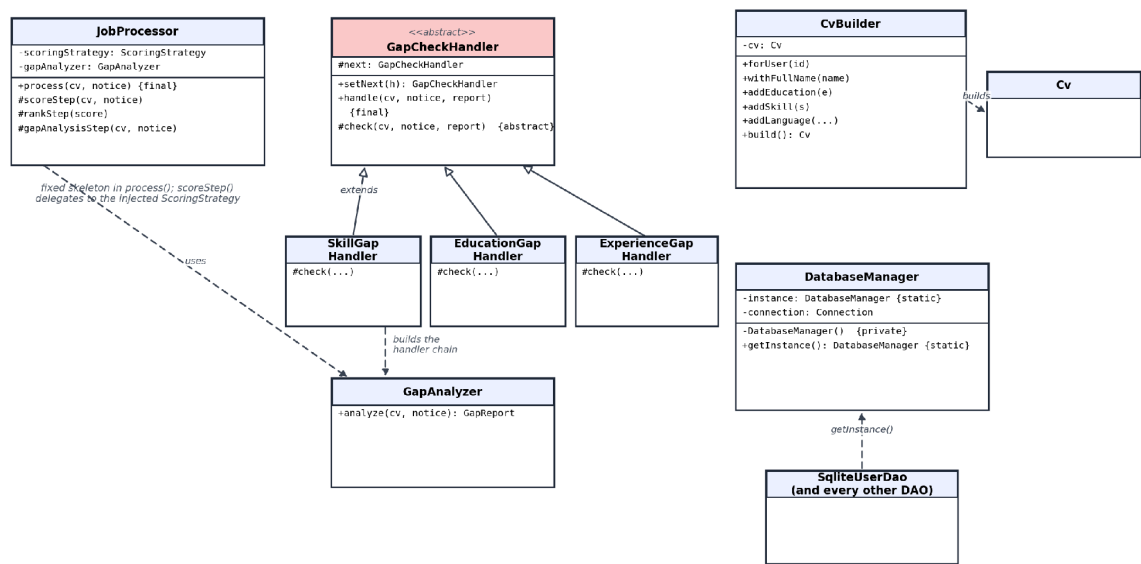


Figure 2 — Template Method (the scoring/ranking pipeline), Chain of Responsibility (gap checks), Builder (CV construction), and Singleton (the shared database connection).

3. Entity-Relationship Diagram

All 13 tables, at the column level. The schema clears the “minimum 4 meaningful entities” bar by a wide margin, but that's a consequence of modeling a CV and a job notice with their real sections (education, experience, skills... / responsibilities, requirements, per-role positions) as proper child tables rather than flattening everything into a couple of wide tables with unstructured text columns. That structure is also what lets the matching and gap-analysis algorithms reason about specific parts of a CV instead of one undifferentiated blob.

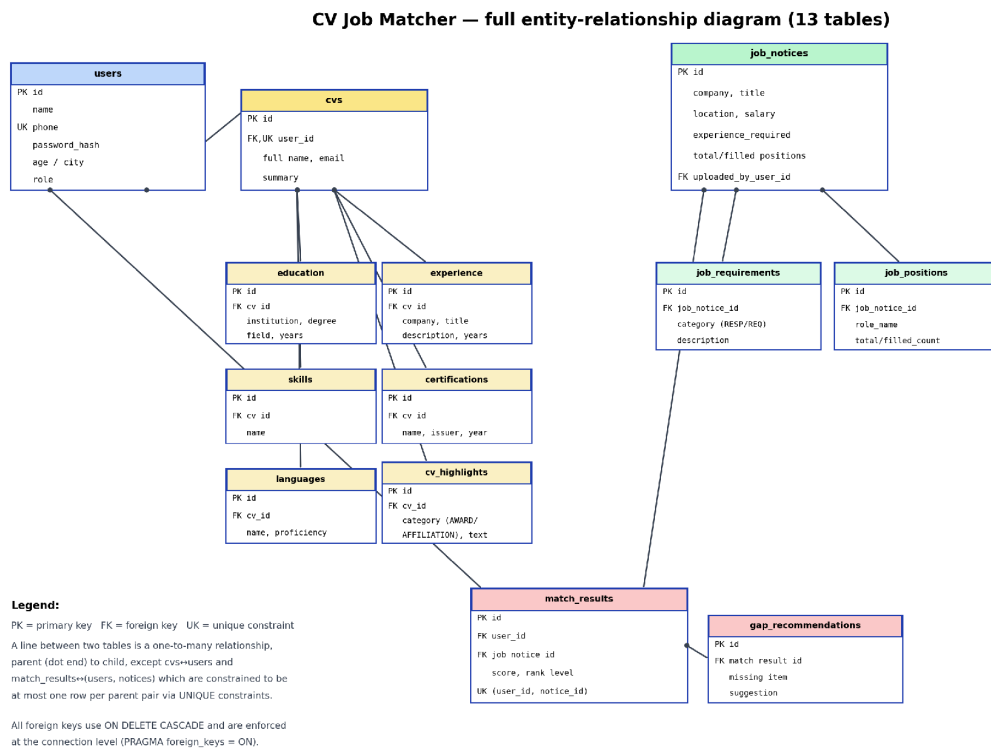


Figure 3 — Full schema. Dot markers show the “one” side of each one-to-many relationship; every foreign key cascades on delete and is enforced at the connection level.

3.1 A few schema decisions worth calling out

- cvs.user_id is both a foreign key and UNIQUE — the database itself enforces “at most one CV per user,” not just application code.
- match_results has a UNIQUE (user_id, job_notice_id) constraint, so re-ranking a notice updates the existing row (and its gap_recommendations, which are deleted and re-inserted together) instead of accumulating duplicate rows every time the matches screen is refreshed.
- job_requirements and job_positions both hang off job_notices independently — a notice's responsibility/requirement bullets and its per-role headcount are separate concerns and don't need to know about each other.

- users.phone is UNIQUE and doubles as the login identifier; login itself checks name + phone + password together, not just phone + password.